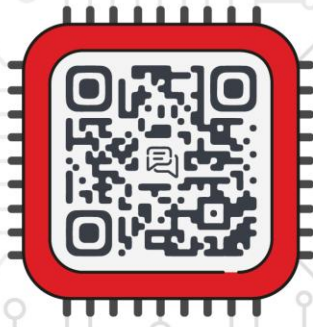




نظري

11

3

1650 sp

كلية الهندسة المعلوماتية

السنة الرابعة

توصيف المتطلبات الغير وظيفية

د . عمار جوخدار



محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 2026/4/13

برمجة تفرعية

- ستكون هذه المحاضرة بداية دخولنا في تفاصيل كل متطلب من المتطلبات غير الوظيفية التي تم تعدادها سابقاً وسنتعرف على معنى كل متطلب منهم وتوصيفه.
- و نجيب على سؤال "ما هي المشكلة" لنتمكن لاحقاً من تعلم كيفية تنفيذ المتطلب والإجابة على سؤال "كيف تحل المشكلة".

المتطلبات الغير وظيفية

1. الأداء :

وهو سرعة النظام و قدرته على تنفيذ المهام تحت ضغط بيانات و مستخدمين.
متى نقول عن برنامج أنه سريع ؟

لا يوجد برامج سريعة وبرامج غير سريعة يتم تحديد ذلك من خلال اتفاق بين البائع والشاري حيث أنه يوجد سرعات غير ممكنة (الاتفاق مهم) بالإضافة أنه لا يوجد معايير عالمية لتحديد سرعة النظام لأن سعرها يختلف، أيضاً توجد عدة أجوبة مختلفة باختلاف وجهات النظر لنتعرف عليها:

■ من وجهة نظر الجهة الطالبة (الزبون) Requirement:

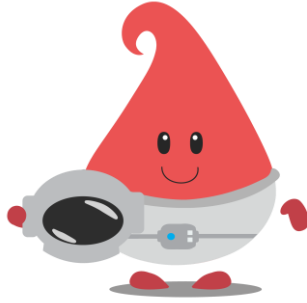
يريد الزبون من النظام أن يحقق عدة أمور ليكون أدائه عالياً:

1. أن ينفذ عدد محدد من المهام خلال زمن محدد: مثلاً ينفذ 10 عمليات خلال ثانية.

- هل تحدد سرعة عالية لكل المهام؟ لا، تحدد سرعة أداء المهمة بحسب عدد مرات تكرارها فكلما ارتفع عدد تكرارات المهمة زادت أهمية تنفيذها بسرعة (مثل ترفيع 6000 طالب بالوقت نفسه أو إدخال علاماتهم بالوقت نفسه).
- بينما في العمليات ذات التكرار القليل أو التي تتكرر بفترات زمنية متباعدة نسبياً فلا داعي لزيادة سرعة تنفيذها بدرجة عالية (تغيير كلمة سر حساب فيسبوك - تسجيل طالب كل 5 دقائق - جلب تقارير من لوحة الادارة).

2. أن ينفذ مهام محددة على عتاديات محددة:

قد يكون البرنامج سريعاً على كمبيوتر محدد وبطيئاً على آخر ، وذلك حسب عتاد كل منهما ، لذلك يجب تحديد هذه الأجهزة:



- ما هي CPU التي يعمل عليها البرنامج ؟
- كم RAM لدينا ؟
- كم حجم Cache Memory المتاحة ؟
- كم حجم Hard Disk المتاح ؟
- Bandwidth و Latency.
- سرعة Bus.

(وهذه عوامل مهمة جداً في تحديد سرعة البرنامج)

3. أن يتواجد عدد محدود من المستخدمين المتنافسين:

من الضروري وجود عدد محدد من المستخدمين الموجودين بنفس الوقت على النظام ويستخدمونه على التوازي لأداء مهام محددة ، إذ من الطبيعي أن زيادة عدد هؤلاء المستخدمين سيقلل من سرعة النظام.

4. أن تكون قاعدة البيانات بمعطيات ضخمة:

من الضروري جداً امتلاء قاعدة البيانات بعدد ضخم من المعطيات (غالباً يستخدم حجم معطيات يمثل 10 سنوات للأمام) وذلك لتجنب الأداء الوهمي الذي نراه عند عدد قليل من المعطيات .

5. أن يعمل النظام باستجابة خطية:

يجب أن يستجيب النظام خطياً مع زيادة الموارد أي مثلاً في حال كان لدي سيرفرين فيجب أن يعملوا ضعف عمل سيرفر واحد ويجب أن يخدموا ضعف عدد المستخدمين.

مثال :

لنتذكر مسألة البائع الجوال ذات درجة التعقيد 2^n لدينا مخدّم سرعته X يُعالج 40 مدينة بالساعة مخدّم آخر سرعته $2X$ كم مدينة يُعالج بالساعة ؟ 41 مدينة لأن زمن المعالجة ضعف (درجة التعقيد أسية) حيث 2^{n+1} أصبحت 2^n ($2^{41} = 2 \times 2^{40}$).

في حال كانت الاستجابة غير خطية سيكون هنالك استهلاك كبير للموارد .

6. أن يكون هناك سيناريو اختبار محدد (Test Script):

يجب تحديد كيفية الاختبار أي ما هي العمليات التي سينفذها كل مستخدم وما هو زمن الاستجابة المطلوب.

■ من وجهة نظر الجهة الراغبة بالتعاقد:

هناك عدة أمور مطلوب توافرها لدى الشركة المتعاقدة التي ستقوم بتنفيذ المشروع أهمها :
توافر Bench Marking أي توافر تجارب سابقة لدى الشركة مشابهة لهذه التجربة تسمح لها بالتأكد من قدرتها على التجاوب مع متطلبات الأداء وتحقيق السرعة المطلوبة.

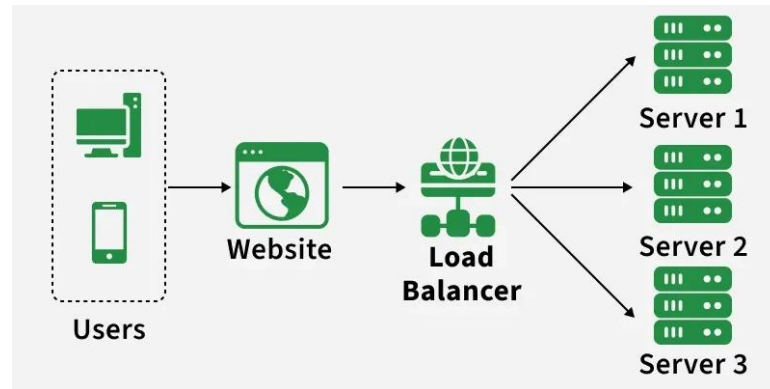
■ مثال :

لدينا برنامج سابق يُعالج 10 آلاف عملية بالثانية نريد معرفة البرنامج المطلوب كم عملية يُعالج هل أكبر أم أصغر من العمليات التي يُعالجها البرنامج السابق.

■ من وجهة نظر المصمم (Designer) :

عليه أن يقوم بعدة أمور بصفته الشخص الذي سيقوم ببناء النظام:

1. يجب تحديد مواصفات البرنامج قبل البدء بتنفيذه و وجود تصوّر كافٍ حوله و التأكد من قدرته على تحقيق الأداء المطلوب.
2. يجب استخدام تقنيات لتحسين الأداء (نماذج تصميم) :
 - pooling: إعادة استخدام الموارد بدل إنشاءها من جديد كل مرة.
 - Caching: تخزين مؤقت للبيانات.
 - Multi-Threading: تنفيذ عدة عمليات بنفس الوقت لتوفير الوقت (لأن البرامج التي يكون فيها أكثر من Thread يكون فيها استغلال لموارد النظام بشكل أفضل).
 - Load Balancing: توزيع الحمل على عدة سيرفرات (كما هو موضح بالصورة) .



■ AOP: فصل المتطلبات غير الوظيفية عن الكود (Aspect Oriented programming).

فائدة ارييسيزية :

- Pooling : بدلاً من إنشاء مسار معالجة (Thread) جديد في المعالج لكل مهمة أو لكل مستخدم جديد يتم تحديد عدد ثابت من الـ Threads الجاهزة . هذه الـ Threads تلتقط المهام، تنجزها، ثم تعود لتلقي مهام جديدة.
- Caching : تخزين البيانات التي يكثر الطلب عليها في ذاكرة الوصول المؤقت لأنها أسرع بكثير من قواعد البيانات العادية
- Load Balancer : أداة تستقبل طلبات الإتصال الكثيفة وتوزعها بذكاء على مجموعة من السيرفرات المتاحة بهدف منع الضغط الزائد على سيرفر واحد لتجنب انهياره،

ولكن كيف يوزع الطلبات بذكاء ؟

في الحقيقية يوجد عدة طرق وخوارزميات لضبط Load balancer فمثلاً لنفترض أن لدينا 3 سيرفرات الأول ضعف قوة الاثنان المتبقيان هنا يمكننا وضع وزن (ثقل) لكل سيرفر فيقوم الـ load balancer بتوجيه طلبان للسيرفر الأقوى مقابل طلب للسيرفر الآخر وهذه إحدى طرق التحكم بالـ Load balancer .

■ من وجهة نظر الجهة المختبرة:

1. على الشخص المختبر القيام ببعض المهام لاختبار البرنامج أنه يؤدي الخدمات المطلوبة :
2. تحتاج آلية لتجهيز بيئة الاختبار المحددة بالمتطلبات ، وهو عمل صعب نسبياً.
2. وضع سيناريوهات ومحاكاة لعدد كبير من المستخدمين: كيف سيتمكن الشخص المختبر من معرفة ما إذا كان هذا البرنامج قادر فعلاً على تخديم العدد المتفق عليه من المستخدمين على التوازي لأداء المهام المتفق عليها بالوقت المحدد ؟
- (فرضاً 10 آلاف مستخدم) باستخدام برامج المحاكاة التي تحاكي أداء عدد المستخدمين المطلوب لقياس زمن استجابة النظام.
3. بعد الانتهاء من اختبار سرعة النظام يجب اختبار صحة النتائج : نلاحظ عند اجراء Bench Marking أنه كلما كان rate الخطأ أعلى كلما زادت سرعة البرنامج !
- أي أنه مثلاً في حال كان السيرفر ممتلئ فعند محاولة دخول أي مستخدم جديد سيتم رفضه فوراً وسيخرج بسرعة أي أن زمن الاستجابة سيزيد لأنه لم يقدّم بأي عمل ، فهل تعتبر هذه سرعة نظام ؟!
- لا، المقياس الأهم للسرعة هو الحصول على نتائج صحيحة.

2. الأمن الوظيفي

يقصد بالأمن الوظيفي : حماية النظام من المستخدمين المعرّفين عليه ولديهم حسابات وصلاحيات دخول (ليس على كل شخص يملك حساب أن يرى ويصل إلى كل شيء في النظام).

يندرج تحت الأمن الوظيفي:

1. التّعرف Authentication.
2. السماحيّة Authorization.
3. المراقبة Monitoring.



■ التّعرف (Authentication) أي معرفة هوية المستخدم :

عند القيام ببرمجة برنامج Server Side فالخطوة الأهم هي معرفة من الشخص الذي يستخدم النظام حالياً ، هناك عدّة طرق لتحقيق التّعرف :

- اسم مستخدم وكلمة مرور.
- Password policy: تتحدد بطول كلمة السر ، صيغتها ، عمرها ، قدرتها.
- تخصيص عنوان IP تقبل منه عمليات الدخول لبعض المستخدمين ، (تحديد مكان محدد يُسمح منه الدخول مثلاً : يُسمح للموظف الدخول للنظام من داخل الشركة فقط).
- تجميد الحساب في حال تكرار الخطأ (إدخال كلمة المرور ب 5 محاولات خاطئة).
- التأكد أن المستخدم إنسان ليس روبوت وأنه الإنسان الذي يدّعي أنه هو وذلك بعدة طرق مثل:

- لوحة مفاتيح غير مرتبة (متغيرة المواقع)

- Captcha: عبارة بها حروف ولا يفهمها سوى الإنسان.



■ **OTP (One Time Password)** : رمز مرور يصل على الجوال لمرة واحدة فقط (يتم إرساله على الجوال لأنه شبكة مفصولة عن اللابتوب في حال كان هناك تجسس على شبكة اللابتوب فلن تتم المهمة بدون OTP).

■ **أسئلة شخصية**: أسئلة عن معلومات خاصة لا أحد يعرفها غيرك ، سبق وأن أدخلت إجاباتها للنظام.

■ **تحديد أوقات دخول بعض المستخدمين (منع دخول موظف للنظام خارج أوقات دوامه).**

■ **السماحية (Authorization) أي إعطاء صلاحيات للمستخدم:**

تحديد صلاحيات المستخدم بعد التعرف عليه ، مثل : قراءة البريد ، تغيير كلمة السر الخاصة بي ، تسجيل الدخول ، تغيير كلمة السر لمستخدم آخر ، تغيير دور مستخدم آخر (حيث كل عملية على النظام تعتبر صلاحية) .

عناصر الأمن الوظيفي :

1. **العملية**: أي إجراء يمكن للمستخدم القيام به (طلب إجازة - رفض إجازة - إنشاء أمر بيع - إنشاء أمر شراء).
2. **المعلومة**: البيانات الناتجة عن تنفيذ العمليات (إجازة - فاتورة).
3. **الدور**: من يستطيع القيام بمجموعة عمليات مترابطة أو الإطلاع على معلومات محددة (مسؤول الدفع - مسؤول البريد الصادر ..)
4. **الصلاحية**: الربط بين (الدور و العمليات و المعلومات) ضمن أزمدة وشروط محددة.

إن عملية توزيع الصلاحيات على المستخدمين لا تتم على المستخدم بشكل مباشر أي أنه لا نقوم بإسناد مجموعة صلاحيات للمستخدم بحد ذاته مباشرة لأنها عملية ضخمة جداً ومعقدة التنفيذ، ما الحل إذا ؟ لنأخذ مثلاً يوضح المشكلة أكثر ثم نتعرف على الحل :

لدينا مستخدم معين ونريد أن نسند له مجموعة الصلاحيات التي من حقّه القيام بها، ولنفرض لدينا 1000 صلاحية سنقوم بإسناد هذه الـ 1000 صلاحية لهذا المستخدم تحديداً.

لنفرض لدينا 100 ألف مستخدم جميعهم لهم نفس الـ 1000 صلاحية ونريد إسناد الصلاحيات لكل مستخدم

◀ 1000 صلاحية * 100000 مستخدم = مليون عملية اسناد !!

لا يوجد Administrator يستطيع تعريف هذا العدد من المستخدمين والقيام بكل عمليات الإسناد لهم بشكل فردي لأنها كما ذكرنا عملية ضخمة جداً .

الحل :

1- **جميع المستخدمين في أدوار (Roles)** بحيث كل دور يضم مجموعة من المستخدمين يحتاجون نفس الصلاحيات

مثل: مستخدم عادي، مستخدم VIP ، Admin ولنفرض حصلنا على 10 أدوار لكل دور صلاحياته الخاصة.

2- **جميع الصلاحيات في مجموعات "groups"** حيث كل مجموعة تسند إلى دور أو عدة أدوار وكل دور يأخذ مجموعة صلاحيات أو أكثر، مثل :مجموعة أعمال يومية - تقارير - إضافة منتج إلى السلة - حذف منتج من التطبيق

لنفرض حصلنا على 30 مجموعة توزع على الأدوار .

◀ لنقل أننا نريد إسناد 30 مجموعة صلاحيات إلى الـ 10 أدوار

30 مجموعة * 10 أدوار = 300 عملية إسناد فقط ! وهو أقل بكثير من 100 مليون.

Role Based Security هي توزيع الصلاحيات في مجموعات والمستخدمين في أدوار ثم إسناد الصلاحيات للمستخدمين ، فعند تعريف مستخدم جديد يجب أن تسند له Role وعند إسناد هذا الـ Role للمستخدم سيأخذ صلاحياته الممنوحة لهذا الـ Role

أمثلة عن صلاحيات العمليات (العمليات التي يقوم بها الدور) :

1. مسؤول الدفع : يسدد مبلغ مستحق على الشركة.
2. مسؤول القبض : يستطيع قبض مبلغ مستحق.
3. مسؤول البريد الوارد : يستطيع إنشاء بريد وتعديله.
4. مسؤول البريد الصادر : يستطيع طباعة بريد صادر.

أمثلة عن صلاحيات المعلومات (البيانات التي يصل لها الدور) :

1. مسؤول القبض يطلع فقط على عمليات القبض التي تمت على صندوقه.
2. مسؤول البريد الصادر يطلع فقط على البريد الخاص بمجموعته.

أمثلة عن أزمدة الصلاحيات وشروطها :

1. يمنع تعديل إجازة موظف بعد الموافقة عليها من المدير.
 2. يمكن لأمين المستودع استرداد بضاعة فقط في حال كانت قد خرجت من مستودعه.
 3. يمكن لأمين المستودع تسليم البضائع في حال وجود أمر تسليم فقط .
 4. يمكن للموظف طلب إجازة في أي وقت يريده لكن لا يمكنه اجراء عملية بيع إلا في أوقات الدوام.
- من الضروري وجود محرك يضبط الأمن مرتبط بتسلسل بالعمليات (نظام داخلي يراقب التنفيذ).

3. المراقبة (Monitoring) :

متابعة ما يحدث داخل النظام بشكل مستمر لاكتشاف أي مشكلة أو سلوك غير طبيعي، أي مراقبة المستخدمين والعمليات والأداء.

أحد الأمثلة على المراقبة:

في تطبيقات Server Side المؤسساتية يُمنع إجراء عملية Delete من قاعدة البيانات أي أنه إذا حاول مستخدم حذف حسابه فإن الحذف سيكون منطقي للمستخدم فقط وليس فيزيائي لقاعدة البيانات أي أن الحساب يبقى في قاعدة البيانات دائماً وللأبد (soft delete).

4. المرونة (Flexibility) :

يُقصد بالمرونة تعديل سلوك البرمجية دون العودة للبرمجة وتغيير الكود.
إلى أي درجة يجب أن تكون مرونة البرنامج ؟

يجب تحديد مستوى المرونة المطلوب من البداية في مرحلة المتطلبات والتحليل لتحديد التعديل هل سيكون على مستوى الإجراءات أم الصلاحيات أم الحسابات ، وهي مُهمّة المُحلّل.

يتم تحقيق المرونة برمجياً عبر تحويلها إلى :

1. إعدادات (Settings) : مثل تغيير Time Zone أو تغيير قيمة الضريبة.
2. صلاحيات (permissions) : مثل تفعيل أو الغاء ميزة من لوحة التحكم بحيث يستطيع المدير فعل ذلك دون برمجة.

يفيدنا استخدام (AOP) في إضافة ميزات جديدة دون تعديل الكود القديم ، بينما عجزت مرونة التحليل عن إضافة سمات جديدة دون تعديل الكود القديم (كسر النظام) فنتجنب حدوث خطأ (Regression).

كل ما زادت المرونة سيزيد تعقيد النظام وتكلفته بينما ينخفض أدائه ونوعيته.

فائدة اربسيزية:

المُحلّل : هو الشخص المسؤول عن تحديد مرونة البرنامج ، حيث تعتمد مرونة أي برنامج على مدى خبرة المُحلّل وقدرته على تحليل الأنماط والمسارات السابقة لتحقيق أفضل توقع عن مسار الأمور للأمام والتعديلات المتوقعة كما من المهم جداً فهمه للأعمال Business ، ودرايته بالقوانين وإلمامه بكافة أقسام المؤسسة الطالبة للبرنامج وفهم علاقة الأقسام ببعضها ويملك فكرة عن المشاكل السابقة لتجنبها في المستقبل و هو الشخص الذي يعلم سير مراحل العملية process من أول مرحلة إلى آخر مرحلة بينما يلتزم باقي أعضاء الفريق كل بقسمه (مرحلته).

عمل المصمم هو الأهم بحيث نصل إلى برنامج يكون المرجع الأول لعمل هذه المؤسسة.

5. Audet, Alert, Tracing :

نريد مراقبة النظام ومعرفة ما يحدث داخله بالتفصيل لكن بطريقة منظمة ومدرسة.

يجب تحديد من البداية الطبقات التي ترغب بالتنصت عليها ومراقبتها في مرحلة وضع المتطلبات و تحديد المعلومات المراد جمعها (وقت العملية - حالتها - من قام بها).

أمثلة على الطبقات: واجهة المستخدم - منطق العمل (Business Logic) - قاعدة البيانات (DataBase DB).

مثلاً : مراقبة عمليات الدفع التي تتم في BL أو مراقبة الاستعلامات في DB .

■ تفيدنا المعلومات التي نحصل عليها من عملية المراقبة والتنصت في:

1. Logging : تخزين المعلومات بهدف استخدامها لاحقاً.
2. Tracing (تتبّع) : نستخدم Logs لمتابعة العملية المخزنة خطوة بخطوة.
3. Alert (التنبيه) : تحليل فوري للبيانات وإرسال تحذيرات في حال حدوث شيء خطير (محاولة اختراق - عدد محاولات دخول كثيرة).

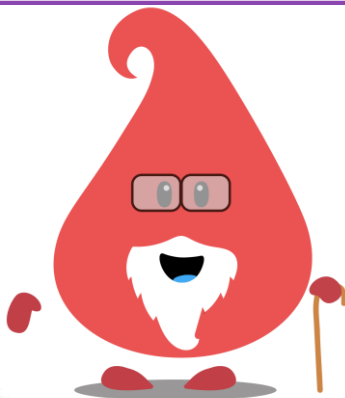
AOP (Aspect-Oriented Programming): هو من أهم نماذج التصميم المفيدة للتنبؤ بحيث تضيف مراقبة من الخارج دون تعديل الكود (مثلًا تضيف Logging لكل العمليات دون تعديل كل دالة).

فائدة ارييسيزية :

- Logging : هو تسجيل العمليات والتفاصيل ليتم مراجعتها لاحقاً وذلك يساعد على معرفة الأخطاء في الكود .
مثال : نضع في Log catch يُسجل حالة الخطأ التي ظهرت ولكن لا نعيدها للمستخدم بل نظهر له حدث خطأ غير معروف بينما إن عُدنا لملف ال logs يمكننا معرفة الخطأ وحله .
- Tracing التتبع: هو متابعة العملية خطوة خطوة لمعرفة كيف حصلت
مثال : قام المستخدم بضغط زر التحويل ثم خصم المال ثم فشلت العملية.
- Alert التنبيه : إرسال تحذير فوري عند حدوث مشكلة أو خطأ غير متوقع
مثال : 10 محاولات تسجيل دخول فاشلة أو EXCEPTION غير متوقع .
- Audit التدقيق : تسجيل كل عملية حصلت بهدف المراجعة لاحقاً
مثال : المستخدم Kareem قام بتحويل 100\$ الساعة 10 صباحاً للمستخدم Abd .

أي بالمختصر :

- الـ Log هو مذكرات المبرمج يسجل فيها الأخطاء والمعلومات ولا يظهرها للمستخدم .
 - الـ Trace هو خريطة الرحلة والتفاصيل التي مرّ فيها الطلب والعمليات التي طرأت عليه .
 - الـ Alert هو تنبيه فوري عند تحقق شرط معين يدل على مشكلة..
 - الـ Audit هو سجل المحكمة ويكون غالباً مفيد للأدمن أكثر من المطور حيث يمكنه متابعة كل العمليات التي يقوم بها المستخدمين.
- وبالتالي فإن هذه المفاهيم الأربعة تكمل بعضها داخل أي نظام برمجي. فالـ Logging يُساعد المطور على فهم الأخطاء، والـ Tracing يوضح مسار العملية خطوة بخطوة، والـ Alerting يُنبه الفريق عند حدوث مشكلة، بينما الـ Audit يُوفر سجلاً موثقاً لمراجعة تصرفات المستخدمين. وعند استخدامها معاً يصبح النظام أسهل في المراقبة، وأسرع في اكتشاف المشاكل، وأكثر أماناً وتنظيماً.



Don't wait for it.
Create it .

6. إدارة المناقلات (ACID)

المناقلة : عملية تجارية كاملة يجب أن تُنفَّذ بشكل صحيح أو لا تُنفَّذ أبداً.

لكي نسمي العملية مناقلة يجب أن يتوفّر فيها 4 خصائص:

- Atomicity (الذرية): أن تتم جميع العمليات الجزئية معاً أو لا تتم نهائياً.
- Coherence (التماسك): أن تبقى البيانات صحيحة ومنطقية ومنسجمة قبل وبعد العملية بغض النظر عن نجاح أو فشل المناقلة.
- (منع المستخدم من حذف عملية والنظام لم يحذفها بعد من قاعدة البيانات أو منع إضافة مناقلة لم يطلبها المستخدم) .
- Isolation (العزل): أن تكون نتيجة المناقلة مستقلة عن جميع المناقلات الأخرى التي تعمل معها على التوازي.

مثال :

لدى سعيد حساب في البنك برصيد 500 يريد سحب 100 وزوجته تريد سحب 100 أي لدينا 2 threads يقومون بنفس العملية على التوازي (أي بفرض كلاهما طلبا السحب بنفس اللحظة) :
فيكون تسلسل العمليات لدى كل منهم كالآتي (وعلى التوازي) :

1- اقرأ رصيد الحساب فيكون 500

2- اخصم 100 من الرصيد فيبقى 400

3- عدّل القيمة الجديدة للرصيد لتصبح 400

فيكون الرصيد النهائي في المحفظة هو 400 وهذا غير صحيح لأنه تم سحب 200 إذاً عملية العزل غير محققة وتسبب مشاكل مالية .

العزل هو أن لا تتنفذ مناقلتان على التوازي بنفس الوقت أي أن العملية تنتظر العملية الأخرى حتى تنتهي.

■ Durability (الديمومة):

نتيجة المناقلة لا تتغير أبداً لا بحالة النجاح ولا الفشل ، أي في حال نجحت عملية ستبقى محفوظة بالنظام ولا يمكن إرسال تقرير بفشلها حتى لو تعطل الخادم.

مثلاً : تم تحويل مبلغ مالي والعملية تمت بنجاح ، حتى لو تعرّض الخادم لعطل العملية تبقى محفوظة ولا يمكن نفي العملية أبداً .

إذاً المناقلة هي مصطلح تجاري يجب أن يحقق جميع هذه الخصائص لأن نتيجة المناقلة سوف يتم بناء عليها مناقلات أخرى .

مثال عملية شراء أسهم :

نقل المال من حساب الشاري للبائع ثم نقل الأسهم من البائع للشاري ، ففي حال انتقال المال يتم نقل الأسهم وإلا لا يتم نقل شيء، ولكن في حال نقل المال وتمت عملية نقل الأسهم لا يجب على الشاري القيام بعملية RollBack

يقصد بإدارة المناقلات:

الآلية التي يتم من خلالها تحديد كيفية بدء تنفيذ وإنهاء المعاملات (Commit / rollback) لضمان تحقيق خصائص ACID.

■ أنواع إدارة المناقلات Transactions

1. User Managed Transactions (UMT):

- تُدار المناقلة يدوياً من قبل المُبرمج أي هو المسؤول عن تحديد بدايتها ونهايتها وتنفيذها.
- تُتيح للمُبرمج تحكم كامل ودقيق لكنها عرضة أكثر للأخطاء وتكرار الكود وزيادة تعقيده.

2. Container Managed Transactions (CMT):

- تُدار المناقلة تلقائياً من قبل البيئة (مثل Spring).
- المُبرمج لا يحدد يدوياً البداية والنهاية بل يقوم بتحديد سياسة المناقلة (Transaction Policy) ويقوم النظام بتنفيذ الممناعة تلقائياً وفي حال نجحت الدالة يطبق Commit وإلا Rollback.
- تُنتج كود أقل تعقيد وأكثر نظافة كما تُقدم تكامل ممتاز مع AOP.
- لكن فيها تحكم أقل دقة من قبل المُبرمج ويعتمد على ال Framework .

يطلق على (CMT) معنى المناقلات التصريحية، وهي الأسهل تحقيقاً والأكثر مرونة ويندرج ضمنها التصريحات التالية:

ما هي التصريحات : هي سياسات تُحدد كيفية تعامل التابع (الدالة) مع المناقلة المرغوبة التي تم تحديدها بناءً على التابع.

◀ التصريحات : هي إخبار كل دالة كيف تتعامل مع المناقلة .

1. Required:

عندما يقوم التابع بطلب مناقلة بإمكانه الانضمام لمناقلة أخرى مفتوحة أي يتم تنفيذه بنفس المناقلة مع التابع الذي قام باستدعائه , وإذا لم يكن هنالك مناقلة مفتوحة أي التابع الذي قام باستدعائه non transactional عندها بإمكانه فتح مناقلة جديدة لنفسه.

- أي في حال كان التابع الذي استدعاه يملك مناقلة : ينضم لها لتنفيذها معاً
- وإذا كان لا يملك مناقلة : يفتح مناقلة جديدة له .

2. Not Required:

التابع لا يحتاج مناقلة ولا يتأثر بوجودها أو عدمه.

3. Required New:

عندما يقوم التابع بطلب مناقلة يتم فوراً فتح مناقلة جديدة مستقلة له بغض النظر عن التابع الذي استدعاه سواء له مناقلة أم لا. (لا يحتاج لمناقلة ولا يضر وجودها)

- في جميع الحالات يتم فتح مناقلة جديدة له .



4. Supported :

عندما يقوم التابع بطلب مناقلة بإمكانه الانضمام لمناقلة مفتوحة للتابع الذي قام بإستدعائه ويتم تنفيذها معاً أما إذا كان التابع الذي استدعاه لا يملك مناقلة (non transactional) فلا يتم فتح مناقلة جديدة للتابع.

- التابع الذي استدعاه يملك مناقلة : ينضم لها ليتم تنفيذها معاً
- التابع الذي استدعاه لا يملك مناقلة : لا يتم فتح مناقلة للتابع

5. Not Supported :

التابع لا يدعم المناقلة لذلك يجب عدم استدعائه بمناقلة وفي حال كان التابع الذي قام باستدعائه لديه مناقلة مفتوحة يتم تعطيلها حتى ينهي التابع عمله ثم يتم استئناف عملها.

- أي في حال كان التابع الذي استدعاه يملك مناقلة : يُوقّفها مؤقتاً حتى ينتهي ثم تكمل .
- التابع الذي استدعاه لا يملك مناقلة : ينفذ التابع عمله بشكل طبيعي جداً (بدون أي مناقلة لأنه لا يدعمها).

6. Never :

عند استدعاء التابع مع وجود مناقلة مفتوحة يظهر Exception.

7. Mandatory :

عند استدعاء التابع مع عدم وجود مناقلة مفتوحة مسبقاً يظهر Exception.

- أي أن التابع يحتاج مناقلة مفتوحة.

ملخص التصريحات :

النوع	وجود مناقلة مفتوحة	عدم وجود مناقلة
Required	ينضم لها	ينشئ مناقلة جديدة
Not Required	لا يتأثر بوجودها	يعمل بدونها
Required New	ينشئ مناقلة جديدة	ينشئ مناقلة جديدة
Supported	ينضم لها	يعمل بدونها
Not Supported	يعطلها ويوقّفها مؤقتاً	يعمل بدونها
Never	يظهر خطأ Exception	يعمل بدونها
Mandatory	ينضم لها	يظهر خطأ Exception